



Lab Title: Cross-Site Scripting (XSS)

Student Name: Nebeuwa Ifeanyichukwu Raphael

Student ID: 202520850LE

Course Name: BVWS103 – Owasp Top 10 Vulnerabilities
Exploitation Techniques

Instructor Name: AMMAR AMINU

Date of Submission: 01/02/2026

Executive Summary

In this lab, I worked on understanding Cross-Site Scripting (XSS), which is a common security issue in web applications. I created a simple web form that accepted user input and displayed it on a webpage. At first, the application did not properly handle the input, which allowed me to inject a script that ran in the browser. This showed me how attackers can take advantage of weak input handling. After that, I fixed the problem by sanitizing the input so that harmful scripts could not run. This lab helped me understand both how XSS attacks happen and how they can be prevented.

Lab Objective

The main objective of this lab was for me to learn what Cross-Site Scripting (XSS) is and how it affects web applications. I also wanted to understand how attackers use XSS to execute malicious scripts in users' browsers. Another important goal was to learn how to prevent XSS by properly validating and encoding user input before displaying it on a webpage.

Lab Environment

I carried out this lab on my local computer using a simple PHP web application. I used a local web server to run the PHP files and tested everything through a web browser. The application was made up of basic PHP pages and HTML forms that accepted and displayed user input. This simple setup made it easy for me to see how XSS works and how applying security fixes changes the behavior of the application.

Task 1: Exploit Cross-Site Scripting (XSS)

Step 1: Set Up the Vulnerable Web Application

- I created a new folder (Lab10_XSS)

```
Session Actions Edit View Help
(raph@kali)-[/opt/lampp/htdocs]
$ mkdir lab10_xss
mkdir: cannot create directory 'lab10_xss': Permission denied

(raph@kali)-[/opt/lampp/htdocs]
$ sudo mkdir lab10_xss
[sudo] password for raph:

(raph@kali)-[/opt/lampp/htdocs]
$ cd lab10_xss

(raph@kali)-[/opt/lampp/htdocs/lab10_xss]
$ ls
```

- Inside the folder, I created the following files index.php, xss_form.php and xss_output.php

```
(raph@kali)-[/opt/lampp/htdocs/lab10_xss]
$ sudo nano index.php

(raph@kali)-[/opt/lampp/htdocs/lab10_xss]
$ sudo nano xss_form.php

(raph@kali)-[/opt/lampp/htdocs/lab10_xss]
$ ls
index.php  xss_form.php

(raph@kali)-[/opt/lampp/htdocs/lab10_xss]
$
```

Step 2: Implemented the Vulnerable Code

- **index.php:** This code contains a simple input form where users can submit their names.

```
raph@kali: /opt/lampp/htdocs/lab10_xss
Session Actions Edit View Help
GNU nano 8.6 index.php
<form action="xss_form.php" method="post">
  <label for="name">Enter your name:</label>
  <input type="text" id="name" name="name" required><br>
  <button type="submit">Submit</button>
</form>
```

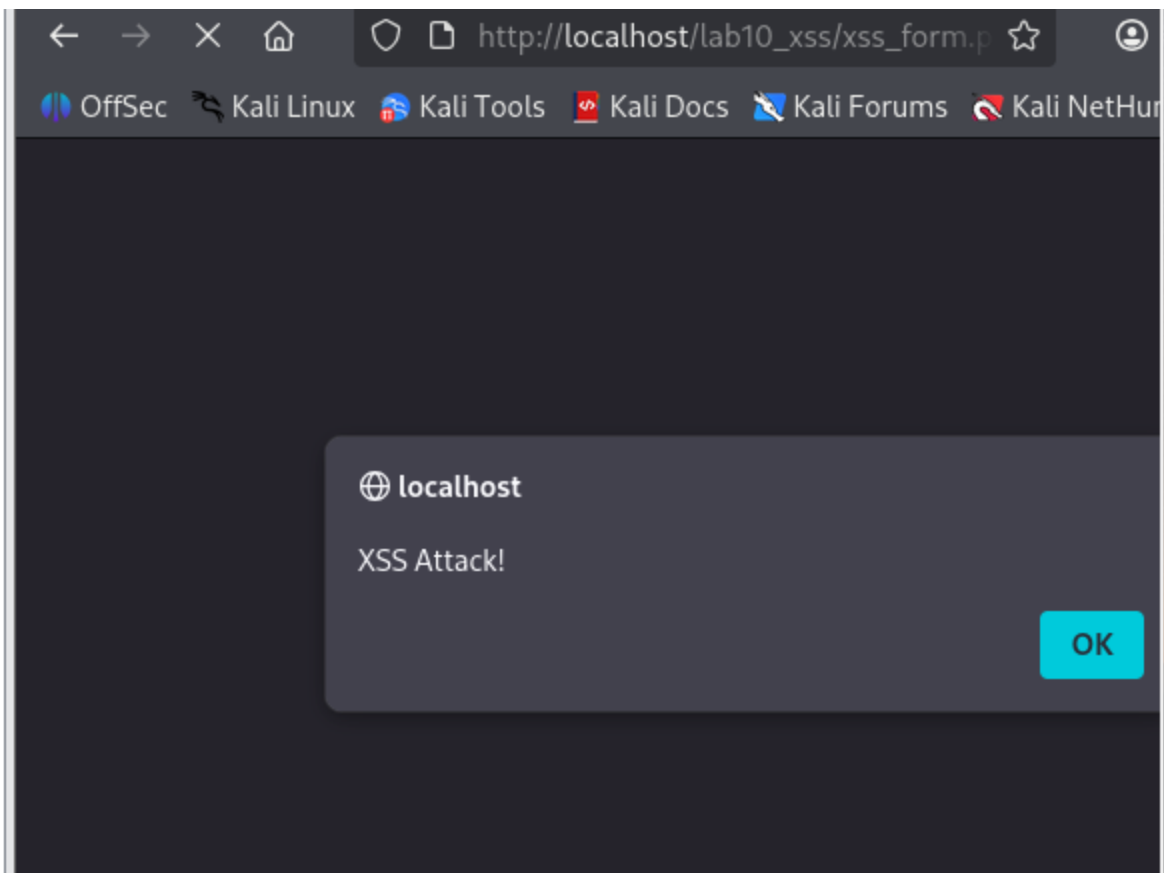
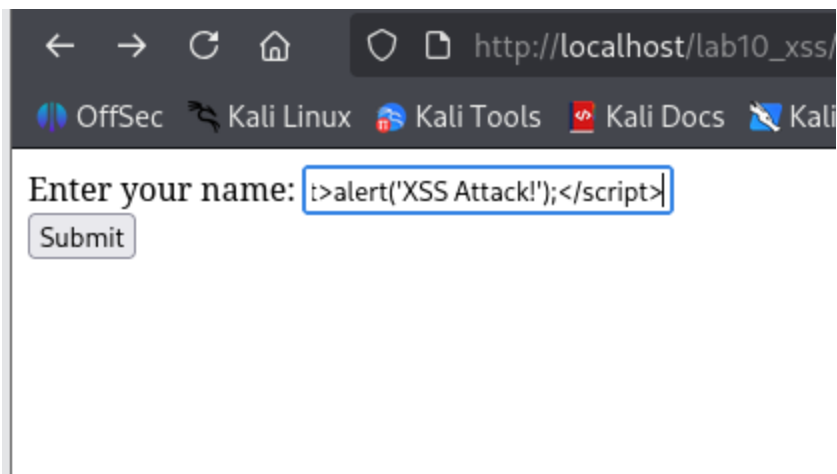
- **xss_form.php:** This code takes user input and submits it without validating or sanitizing it, creating a potential XSS vulnerability.

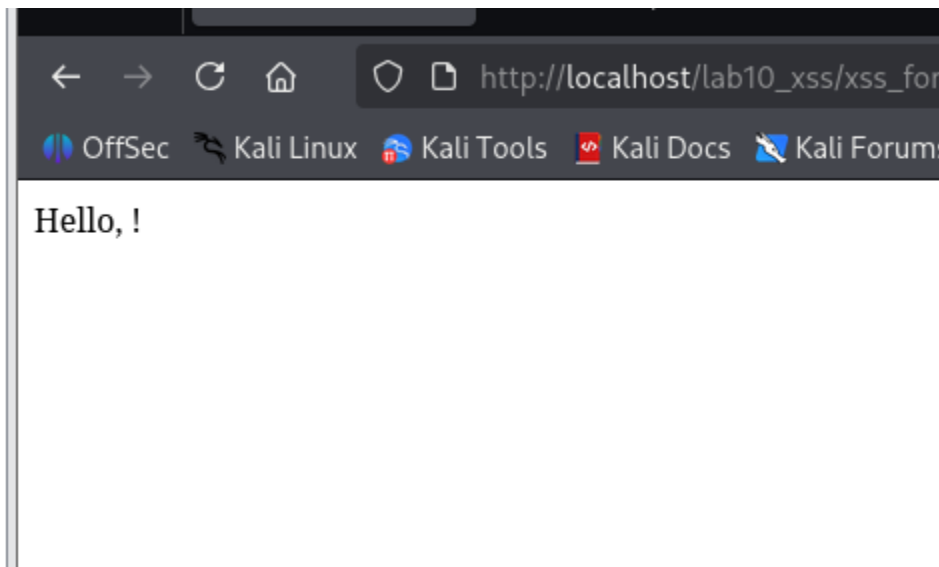
```
raph@kali: /opt/lampp/htdocs/lab10_xss
Session Actions Edit View Help
GNU nano 8.6 xss_form.php
<?php
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $name = $_POST['name']; // Directly using user input without s
    // Vulnerability: XSS
    echo "<p>Hello, $name!</p>"; // User input is directly display
}
?>
```

Step 3: Exploit the Vulnerability

- I submitted a malicious payload in the index.php page.

```
<script>alert('XSS Attack!');</script>
```





Reflection Questions:

- **What happens when a malicious script is injected into the input field?**

When a malicious script is injected into an input field and the application does not sanitize it, the script is stored or reflected back and then executed by the user's browser. The browser treats the script as trusted content from the website, so it runs automatically. This can result in pop-ups, page manipulation, data being read from the page, or actions being performed without the user's knowledge.

- **How can this attack be used to steal session cookies or redirect users to malicious websites?**

An attacker can use an XSS script to read session cookies stored in the browser and send them to an external server controlled by the attacker. With these cookies, the attacker may hijack the user's session and access their account without logging in. The script can also force the browser to automatically redirect the user to a fake or malicious website, which may be used for phishing, spreading malware, or stealing sensitive information.

Task 2: Mitigate Cross-Site Scripting (XSS)

Step 1: Implement Input Validation and Output Endcoding

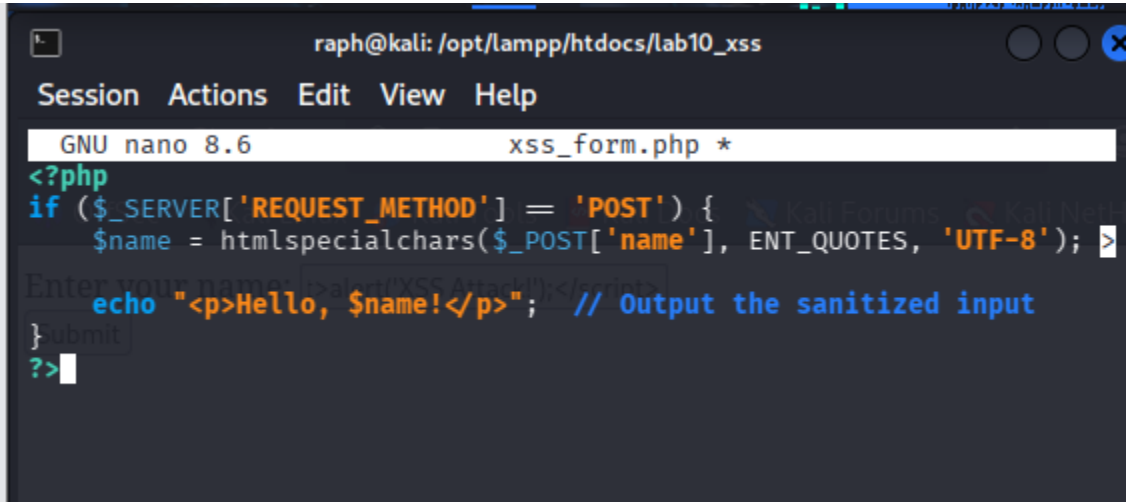
To mitigate the XSS vulnerability, I sanitize user input and ensure that user-submitted content is displayed safely.

- Modifying the xss_form.php page to escape user input before displaying it

```
<?php
if ($_SERVER['REQUEST_METHOD'] == 'POST') {

    $name = htmlspecialchars($_POST['name'], ENT_QUOTES, 'UTF-8'); // Sanitize user input

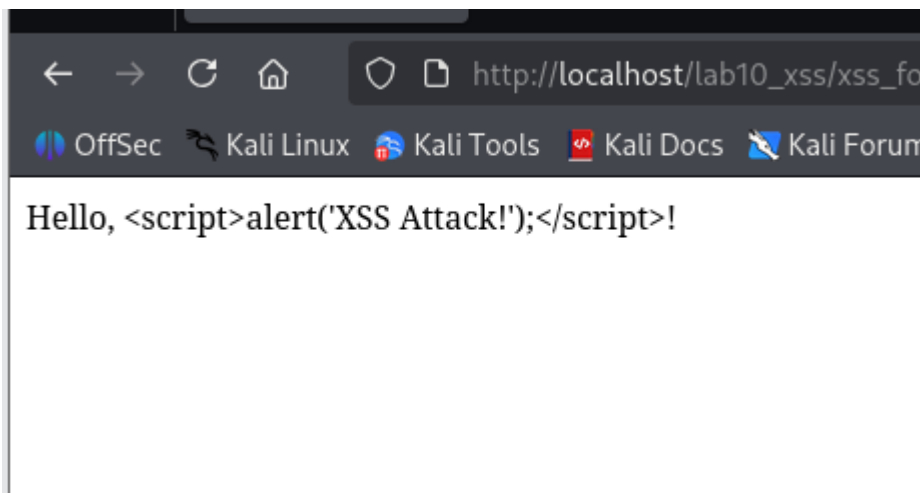
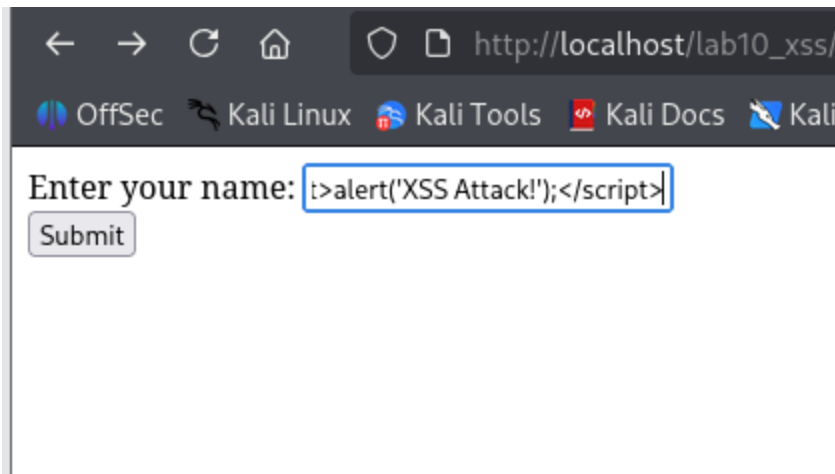
    echo "<p>Hello, $name!</p>"; // Output the sanitized input
}
?>
```



```
raph@kali: /opt/lampp/htdocs/lab10_xss
GNU nano 8.6 xss_form.php *
<?php
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $name = htmlspecialchars($_POST['name'], ENT_QUOTES, 'UTF-8');
    echo "<p>Hello, $name!</p>"; // Output the sanitized input
}
?>
```

Step 2: Test the Mitigated Version

I submitted the malicious payload `<script>alert('XSS Attack!');</script>` again



Reflection Questions:

- How did sanitizing the user input prevent the XSS attack?

Sanitizing the user input prevented the XSS attack by removing or converting special characters that are used to run scripts, such as `<` and `>`. Instead of the browser treating the input as executable code, the input is handled as plain text. This means that even if an attacker enters a script, it will not run and will only be displayed as text on the page.

- What is the importance of using output encoding in preventing XSS vulnerabilities?

Output encoding is important because it ensures that any user-provided data shown on a webpage is displayed safely. It stops the browser from interpreting the data as HTML or

JavaScript. By encoding the output, web applications protect users from malicious scripts, reduce the risk of session hijacking, and help keep the application secure.

Index.php

```
<form action="xss_form.php" method="post">  
  <label for="name">Enter your name:</label>  
  <input type="text" id="name" name="name" required><br>  
  <button type="submit">Submit</button>  
</form>
```

Xss_form.php

```
<?php  
if ($_SERVER['REQUEST_METHOD'] == 'POST') {  
  $name = $_POST['name']; // Directly using user input without sanitizing  
  
  // Vulnerability: XSS  
  
  echo "<p>Hello, $name!</p>"; // User input is directly displayed on the page without  
  escaping  
}  
?>
```

Key Takeways

During this lab, I learned that whenever a web developer or programmer is building an application that collects user data, they must always sanitize user input. It is also important to verify that the code follows security best practices and to ensure that the browser does not interpret HTML or script code as valid input. This helps prevent security issues such as Cross-Site Scripting (XSS) and protects users' data.

Conclusion

In this lab, I learned how Cross-Site Scripting (XSS) vulnerabilities occur and how easily they can be exploited when user input is not properly handled. By testing a vulnerable application and then fixing it through input sanitization and output encoding, I understood the importance of securing user input. This lab helped me realize that small mistakes in code can lead to serious security problems, and applying simple security measures can greatly improve the safety of web applications.